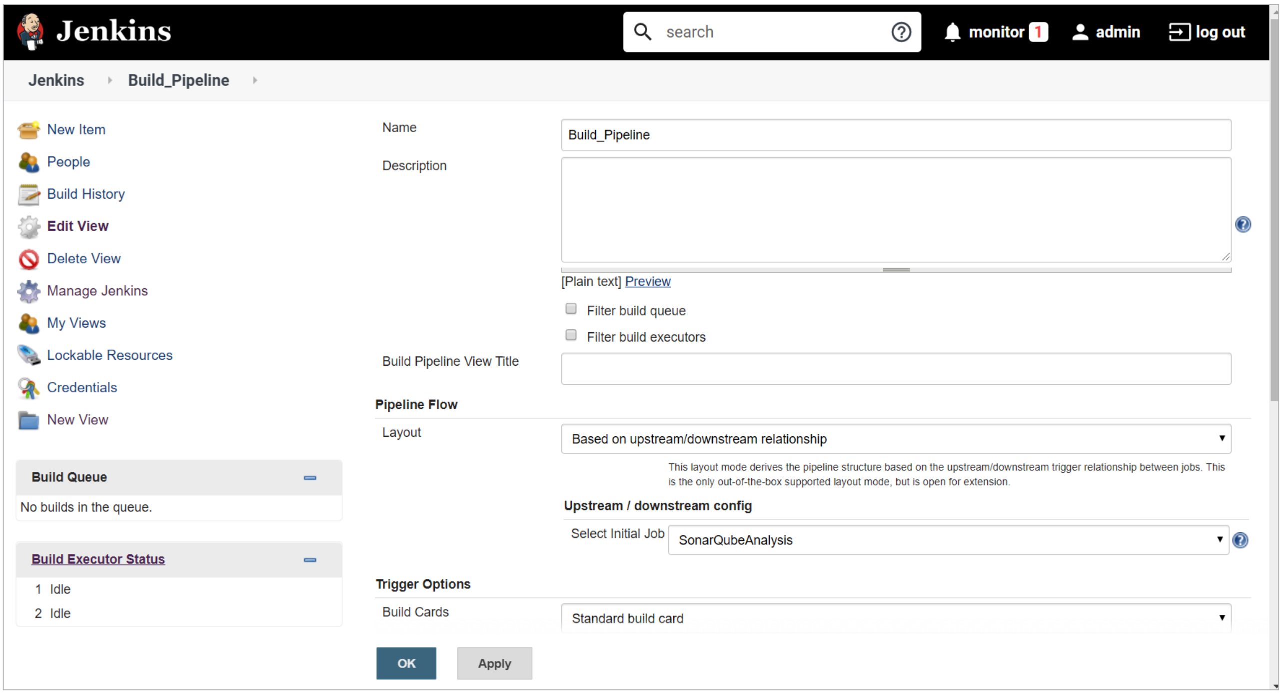

Figure 1: Build pipeline configuration

Pipeline as code helps you fix the above issues as it supports the following:

- Domain-specific language (DSL) helps to create pipelines through the DSL or *Jenkinsfile*.
- Pipeline as a code can utilise all programming concepts.
- Has a common approach and standards for all teams.
- Version control of the pipelines, as it is also a script and managed in version control systems such as SVN or Git.

The pipeline helps to define and implement continuous integration, continuous testing (automated functional testing, load testing, security testing) and continuous delivery using DSL in Jenkins. *Jenkinsfile* contains the script to automate continuous integration, continuous testing, and continuous delivery or it is available in the Jenkins pipeline job.

There are two ways to create a pipeline — Jenkins dashboard and *Jenkinsfile*. It is a best practice, however, to use *Jenkinsfile*. Table 1 explains some basic pipeline concepts.

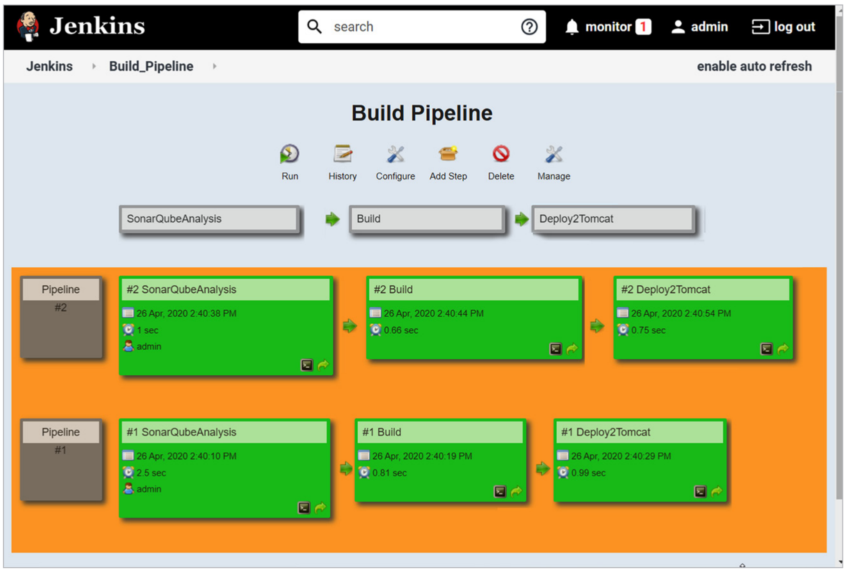

Figure 2: Build pipeline view

Pipeline (declarative)	This models a CI/CD pipeline, or it contains stages and phases of application life cycle management.
Agent or node	This is a system utilised in the controller/master agent architecture of Jenkins. Jenkins execution takes place on the controller/master or agent node.
Stage	This is a collection of tasks; e.g., compilation of source files, unit test execution, and publishing Junit test reports.
Step	This is a task. Multiple tasks make a stage.

Table 1: Pipeline concepts